BZ205 Konu Anlatımı

Sıralama Algoritmaları

Hocanın ders videoları, slaytları ve sınav soruları temel alınmıştır

27 Ekim 2026

3 Sıralama Algoritmaları

Ders kapsamındaki algoritmalar: araya sokma (insertion), seçmeli (selection), kabarcık (bubble), birleşmeli (merge), kümeleme (heap), hızlı (quick) ve radiks (radix). Hepsinin çalışma mantığı farklıdır; her birinin iyi ve kötü çalıştığı durumlar *en iyi / en kötü durum* analiziyle belirlenir.

! Hoca derste vurguladı [video 4 Sıralama 1 56:17]

*"**Sınavlarda genelde alfabe üzerinden soruyorum. Alfabeyi öğrenelim.**"* [video 8 Sıralama 5 22:15]: Türkçe ve İngilizce harfler karışık gelebilir; birleşik sıra:

A B C Ç D E F G Ğ H I İ J K L M N O Ö P Q R S Ş T U Ü V W X Y Z

"Bir kısmınız bunlar hem İngilizce hem Türkçe olduğu için sıralanamaz diye sınav kağıtlarında yazıyor" – yanlış. *"Programa soktum, Ğ, Ç, Ö, Ü'yü Z'den sonra gösterdi"* – *"böyle olmaz"*.

Örneklerde hocanın derste kullandığı dizi kullanılmıştır: **7, 3, 5, 8, 2, 9, 4, 15, 6**.

3.1 Araya sokma sıralama (Insertion sort)

Mantık: Dizi iki parçaya ayrılır: solda sıralı kısım, sağda sırasız kısım. İlk eleman sıralı kabul edilir. Sırasız kısmın ilk elemanı elde tutulur (`ekle`), sıralı kısımla *sondan başa* karşılaştırılır; kendisinden büyük olanlar bir sağa kayarak *yer açar*, boşluğa `ekle` yerleştirilir (iskambil kağıdı dizme gibi).

```
void arayaSokma(int D[], int n) {
  int i, k, ekle;                          // ihmal
  for (i = 1; i < n; i++) {                // 1. eleman yerinde kalir
     ekle = D[i];                          // siradaki eleman elde
     for (k = i-1; k >= 0 && ekle <= D[k]; k--)
        D[k+1] = D[k];                     // buyuk eleman bir saga kayar
     D[k+1] = ekle;                        // bosluga yerlestir
  }
}
```

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

i	ekle	kayan eleman	Dizi
–	–	–	7, 3, 5, 8, 2, 9, 4, 15, 6 (ilk eleman sıralı kabul edilir)
1	3	1	**3**, 7, 5, 8, 2, 9, 4, 15, 6
2	5	1	3, **5**, 7, 8, 2, 9, 4, 15, 6
3	8	0	3, 5, 7, **8**, 2, 9, 4, 15, 6
4	2	4	**2**, 3, 5, 7, 8, 9, 4, 15, 6
5	9	0	2, 3, 5, 7, 8, **9**, 4, 15, 6

i	ekle	kayan eleman	Dizi
6	4	4	2, 3, **4**, 5, 7, 8, 9, 15, 6
7	15	0	2, 3, 4, 5, 7, 8, 9, **15**, 6
8	6	4	2, 3, 4, 5, **6**, 7, 8, 9, 15

Sonuç: 2, 3, 4, 5, 6, 7, 8, 9, 15

- **En kötü durum:** dizi büyükten küçüğe sıralı – her eleman en başa kadar kayar. **Tüm elemanlar eşitse de en kötü durumdur** çünkü koşul `ekle <= D[k]` eşitlikte de döngüye girer [video 5 37:43].
- **En iyi durum:** dizi küçükten büyüğe sıralı – kaydırma satırı hiç çalışmaz.
- Dış for: $1+n+(n-1)=2n$; `ekle` ataması $n-1$; iç döngü toplam $1+2+\cdots+(n-1)=\frac{n^2-n}{2}$ $\Rightarrow T(n)$ ikinci dereceden: $O(n^2)$, en iyi $\Omega(n)$.

Satır satır yürütme zamanı (değişken tanımları ihmal edilmiştir)

Satır	En iyi	En kötü
`for (i = 1; i < n; i++)`	$2n$	$2n$
`ekle = D[i]`	$n-1$	$n-1$
`for (k = i-1; k >= 0 && ekle <= D[k]; k--)`	$2(n-1)$	n^2+n-2
`D[k+1] = D[k]` (kaydırma)	0	$n(n-1)/2$
`D[k+1] = ekle`	$n-1$	$n-1$
$T(n)$	$\mathbf{6n-4}$	$\mathbf{(3n^2+9n-8)/2}$

Zaman ve alan karmaşıklığı

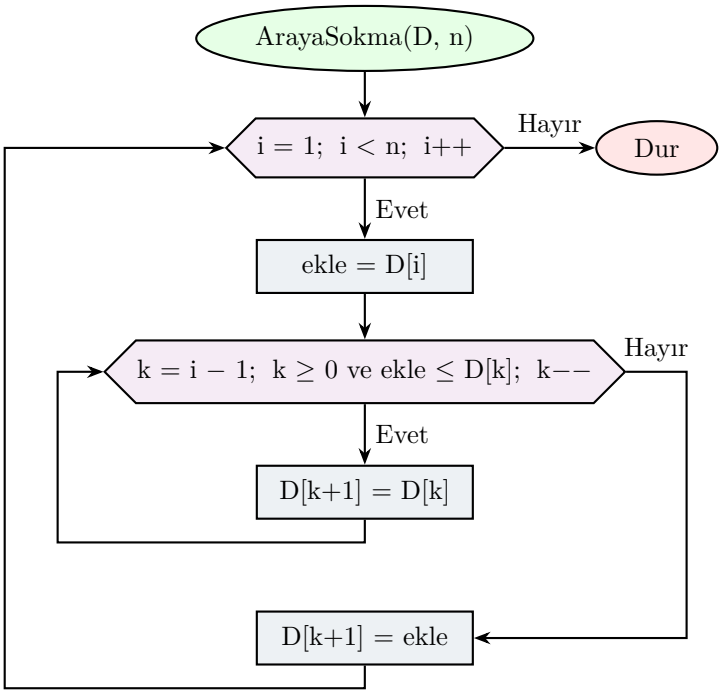
Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n)$	dizi küçükten büyüğe sıralı: iç döngü koşulu ilk kontrolde yanlış, kaydırma yok
En kötü	$O(n^2)$	dizi büyükten küçüğe sıralı *veya tüm elemanlar eşit* (`<=` yüzünden eşitte de kayar)
Ortalama	–	en iyi n, en kötü n^2 dereceli $\Rightarrow \Theta$ verilmez
Ek alan	$O(1)$	sadece `ekle` değişkeni; dizi yerinde sıralanır

Araya sokma – nihai özet

Özellik	En iyi durum	En kötü durum
Eleman karşılaştırması (`ekle <= D[k]`)	$n-1$	$n(n-1)/2$
Kaydırma (`D[k+1] = D[k]`)	0	$n(n-1)/2$
$T(n)$	$6n-4$	$(3n^2+9n-8)/2$
Zaman karmaşıklığı	$\Omega(n)$	$O(n^2)$
Ek alan	$O(1)$	$O(1)$

Akış diyagramı: araya sokma sıralama

İç döngü de kodda `for` olduğu için altıgen döngü sembolüyle çizilmiştir: başlangıç `k = i-1`, koşul `k >= 0 && ekle <= D[k]`, artım `k--`; gövdesi yalnızca kaydırma satırıdır, döngü bitince (Hayır) `D[k+1] = ekle` yapılır.



3.2 Seçmeli sıralama (Selection sort)

Mantık: Dizideki en küçük eleman *seçilir* ve en başa konur; kalanlar içinden en küçük seçilip ikinci sıraya konur…

```
void secmeli(int D[], int n) {
  int i, j, enk, indis;                    // ihmal
  for (i = 0; i < n-1; i++) {
     enk = D[n-1];  indis = n-1;           // EN SONDAKI eleman en kucuk kabul edilir
     for (j = i; j < n-1; j++)
        if (D[j] < enk) { enk = D[j]; indis = j; }
     D[indis] = D[i];                      // bastaki eleman en kucugun yerine
     D[i] = enk;                           // en kucuk basa
  }
}
```

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

i	Seçilen en küçük	Dizi (yer değiştirme sonrası)
0	2	**2**, 3, 5, 8, **7**, 9, 4, 15, 6
1	3	2, **3**, 5, 8, 7, 9, 4, 15, 6
2	4	2, 3, **4**, 8, 7, 9, **5**, 15, 6
3	5	2, 3, 4, **5**, 7, 9, **8**, 15, 6
4	6	2, 3, 4, 5, **6**, 9, 8, 15, **7**
5	7	2, 3, 4, 5, 6, **7**, 8, 15, **9**
6	8	2, 3, 4, 5, 6, 7, **8**, 15, 9
7	9	2, 3, 4, 5, 6, 7, 8, **9**, **15**

Sonuç: 2, 3, 4, 5, 6, 7, 8, 9, 15

! Hoca derste vurguladı [video 5 Sıralama 2 53:01–59:12]

Hocanın kodunda başlangıç en küçüğü *en sondaki* elemandır. Bu yüzden:
En iyi durum: dizi *tersten (büyükten küçüğe)* sıralı veya *tüm elemanlar eşit* – if içi hiç çalışmaz. (Küçükten büyüğe sıralı dizi en iyi durum *değildir*: son eleman en büyük olduğundan ilk karşılaştırmada if içine girilir.)
En kötü durum: son eleman en büyük ve geri kalanı büyükten küçüğe sıralı – if içine her seferinde girilir.
"Verilen koda göre düşünmemiz lazım... En küçüğü en baştakini alırsam senaryoyu ona göre ayarlamam lazım."

İç döngü toplam $(n-1)+(n-2)+\cdots+1=\frac{n(n-1)}{2}$; en iyi ve en kötü durum ikisi de n^2 dereceli $\Rightarrow \Theta(n^2)$.

Satır satır yürütme zamanı (değişken tanımları ihmal edilmiştir)

Satır	En iyi	En kötü
`for (i = 0; i < n-1; i++)`	$2n$	$2n$
`enk = D[n-1]; indis = n-1`	$2(n-1)$	$2(n-1)$
`for (j = i; j < n-1; j++)`	n^2+n-2	n^2+n-2
`if (D[j] < enk)`	$n(n-1)/2$	$n(n-1)/2$
`enk = D[j]; indis = j`	0	$n(n-1)$
`D[indis] = D[i]; D[i] = enk`	$2(n-1)$	$2(n-1)$
$T(n)$	$\mathbf{(3n^2+13n-12)/2}$	$\mathbf{(5n^2+11n-12)/2}$

Zaman ve alan karmaşıklığı

Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n^2)$	if içi hiç çalışmaz (tüm elemanlar eşit; hocaya göre tersten sıralı da)
En kötü	$O(n^2)$	if içi her seferinde çalışır (son eleman en büyük, gerisi azalan)
Ortalama	$\Theta(n^2)$	en iyi ve en kötü aynı derecede; hoca: *"ortalama durum maliyetinden bahsedebiliriz"*
Ek alan	$O(1)$	`enk` ve `indis`; dizi yerinde sıralanır

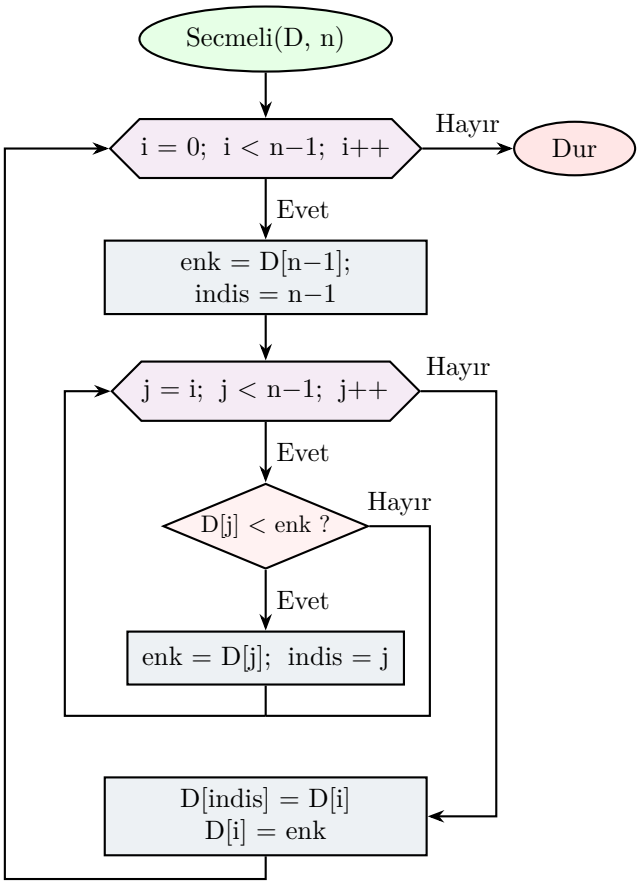
Seçmeli – nihai özet

Özellik	En iyi durum	En kötü durum
Eleman karşılaştırması (`D[j] < enk`)	$n(n-1)/2$	$n(n-1)/2$
if içi (yeni en küçük bulma)	0	$n(n-1)/2$ (üst sınır)
Yer değiştirme (her turda bir)	$n-1$	$n-1$
$T(n)$	$(3n^2+13n-12)/2$	$(5n^2+11n-12)/2$
Zaman karmaşıklığı	$\Omega(n^2)$	$O(n^2)\Rightarrow\Theta(n^2)$
Ek alan	$O(1)$	$O(1)$

Seçmelide en iyi/en kötü durumun inceliği

Tablodaki en iyi ve en kötü $T(n)$, hocanın analizindeki gibi *if içi hiç çalışmaz* ve *if içi her seferinde çalışır* kabulleriyle hesaplanmıştır. Simülasyonla kontrol edildiğinde: tüm elemanlar eşitse if içine gerçekten hiç girilmez; tersten sıralı dizide ise yalnız *ilk turda* hiç girilmez, ilk yer değiştirmeden sonra sondaki eleman değiştiği için sonraki turlarda kısmen girilir. Derece (n^2) her durumda aynı kaldığından sonuç değişmez: $\Theta(n^2)$. Sınavda hocanın kabulünü yazın.

Akış diyagramı: seçmeli sıralama (başlangıç en küçüğü en sondaki eleman)



3.3 Kabarcık sıralama (Bubble sort)

Mantık (deniz dalgası): Her "dalga hareketinde" komşu elemanlar karşılaştırılır, büyük olan sağa geçer. Bir hareket sonunda en büyük eleman en sona yerleşir, küçükler öne yaklaşır (çift yönlü etki). Her harekette iç döngü bir eksik çalışır.

```
void kabarcik(int D[], int n) {
  int gecici, k, hareket;                    // ihmal
  for (hareket = 0; hareket < n-1; hareket++)          // dalga hareketi
     for (k = 0; k < n-1-hareket; k++)                // komsu karsilastirma
        if (D[k] > D[k+1]) {
           gecici = D[k];  D[k] = D[k+1];  D[k+1] = gecici;
        }
}
```

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Hareket	k	İşlem	Dizi (işlem sonrası)
0	0	7 ↔ 3	**3**, **7**, 5, 8, 2, 9, 4, 15, 6
0	1	7 ↔ 5	3, **5**, **7**, 8, 2, 9, 4, 15, 6
0	2	değişim yok	3, 5, 7, 8, 2, 9, 4, 15, 6
0	3	8 ↔ 2	3, 5, 7, **2**, **8**, 9, 4, 15, 6
0	4	değişim yok	3, 5, 7, 2, 8, 9, 4, 15, 6
0	5	9 ↔ 4	3, 5, 7, 2, 8, **4**, **9**, 15, 6
0	6	değişim yok	3, 5, 7, 2, 8, 4, 9, 15, 6
0	7	15 ↔ 6	3, 5, 7, 2, 8, 4, 9, **6**, **15**
1	0	değişim yok	3, 5, 7, 2, 8, 4, 9, 6, 15

Hareket	k	İşlem	Dizi (işlem sonrası)
1	1	değişim yok	3, 5, 7, 2, 8, 4, 9, 6, 15
1	2	$7 \leftrightarrow 2$	3, 5, **2**, **7**, 8, 4, 9, 6, 15
1	3	değişim yok	3, 5, 2, 7, 8, 4, 9, 6, 15
1	4	$8 \leftrightarrow 4$	3, 5, 2, 7, **4**, **8**, 9, 6, 15
1	5	değişim yok	3, 5, 2, 7, 4, 8, 9, 6, 15
1	6	$9 \leftrightarrow 6$	3, 5, 2, 7, 4, 8, **6**, **9**, 15
2	0	değişim yok	3, 5, 2, 7, 4, 8, 6, 9, 15
2	1	$5 \leftrightarrow 2$	3, **2**, **5**, 7, 4, 8, 6, 9, 15
2	2	değişim yok	3, 2, 5, 7, 4, 8, 6, 9, 15
2	3	$7 \leftrightarrow 4$	3, 2, 5, **4**, **7**, 8, 6, 9, 15
2	4	değişim yok	3, 2, 5, 4, 7, 8, 6, 9, 15
2	5	$8 \leftrightarrow 6$	3, 2, 5, 4, 7, **6**, **8**, 9, 15
3	0	$3 \leftrightarrow 2$	**2**, **3**, 5, 4, 7, 6, 8, 9, 15
3	1	değişim yok	2, 3, 5, 4, 7, 6, 8, 9, 15
3	2	$5 \leftrightarrow 4$	2, 3, **4**, **5**, 7, 6, 8, 9, 15
3	3	değişim yok	2, 3, 4, 5, 7, 6, 8, 9, 15
3	4	$7 \leftrightarrow 6$	2, 3, 4, 5, **6**, **7**, 8, 9, 15
4	0	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
4	1	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
4	2	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
4	3	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
5	0	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
5	1	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
5	2	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
6	0	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
6	1	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15
7	0	değişim yok	2, 3, 4, 5, 6, 7, 8, 9, 15

Sonuç: 2, 3, 4, 5, 6, 7, 8, 9, 15

- En iyi durum: dizi küçükten büyüğe sıralı *veya tüm elemanlar eşit* (if içi hiç çalışmaz). En kötü: tersten sıralı (if içine her seferinde girilir).
- İç döngü toplamı $\frac{n(n-1)}{2}$; en iyi ve en kötü fark yaklaşık $3n^2/2$ terimi (yer değiştirmenin 3 ataması) $\Rightarrow \Omega(n^2)$, $O(n^2)$, $\Theta(n^2)$.

Satır satır yürütme zamanı (değişken tanımları ihmal edilmiştir)

Satır	En iyi	En kötü
`for (hareket = 0; hareket < n-1; hareket++)`	$2n$	$2n$
`for (k = 0; k < n-1-hareket; k++)`	$n^2 + n - 2$	$n^2 + n - 2$
`if (D[k] > D[k+1])`	$n(n-1)/2$	$n(n-1)/2$
yer değiştirme (3 atama)	0	$3n(n-1)/2$
$T(n)$	$\mathbf{(3n^2 + 5n - 4)/2}$	$\mathbf{3n^2 + n - 2}$

Zaman ve alan karmaşıklığı

Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n^2)$	dizi küçükten büyüğe sıralı veya tüm elemanlar eşit: if içi hiç çalışmaz ama döngüler yine tam döner
En kötü	$O(n^2)$	dizi büyükten küçüğe sıralı: her karşılaştırmada yer değiştirme
Ortalama	$\Theta(n^2)$	en iyi ve en kötü aynı derecede
Ek alan	$O(1)$	sadece `gecici` değişkeni; dizi yerinde sıralanır

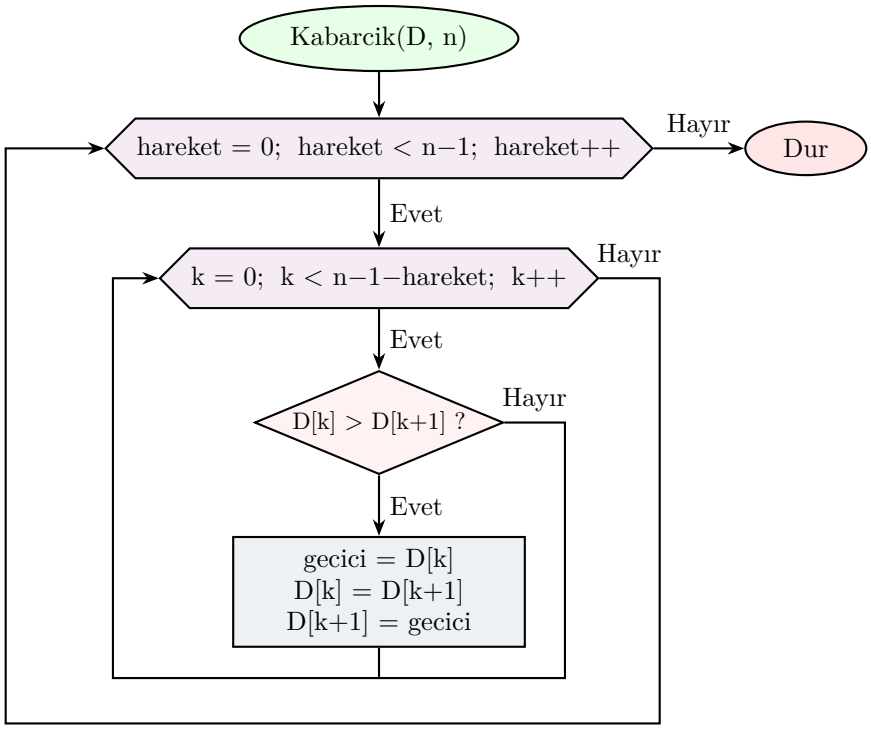
Kabarcık – nihai özet

Özellik	En iyi durum	En kötü durum
Eleman karşılaştırması (`D[k] > D[k+1]`)	$n(n-1)/2$	$n(n-1)/2$
Yer değiştirme	0	$n(n-1)/2$
$T(n)$	$(3n^2+5n-4)/2$	$3n^2+n-2$
Zaman karmaşıklığı	$\Omega(n^2)$	$O(n^2) \Rightarrow \Theta(n^2)$
Ek alan	$O(1)$	$O(1)$

Başka kaynaklarda "en iyi durum $O(n)$" görürseniz

O kaynaklar, bir dalgada hiç yer değiştirme olmazsa döngüden çıkan bir *bayrak* ekler. Hocanın kodunda bu bayrak yoktur; sıralı dizide de bütün dalgalar döner, bu yüzden en iyi durum $\Omega(n^2)$'dir. Bir öğrencinin WhatsApp'ta paylaştığı satır satır tablo da aynı sonuca varır (en iyi ve en kötü ikisi de n^2 dereceli).

Akış diyagramı: kabarcık sıralama



3.4 Birleşmeli sıralama (Merge sort)

Mantık (böl ve fethet): Dizi **pivot = (ilk indis + son indis)/2, aşağı yuvarlanır** ile ikiye bölünür (pivot ve solu sol parçada). Tek eleman kalana kadar bölünür. Sonra sıralı parçalar *ilk elemanları karşılaştırılarak* birleştirilir.

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Bölme aşaması (pivot = (ilk+son)/2 aşağı yuvarlanır, pivot ve solu sol parçaya):

[7, 3, 5, 8, 2] | [9, 4, 15, 6]

· [7, 3, 5] | [8, 2]

· · [7, 3] | [5]

· · · [7] | [3]

· · [8] | [2]

· [9, 4] | [15, 6]

· · [9] | [4]

· · [15] | [6]

Birleştirme aşaması (sıralı parçaların ilk elemanları karşılaştırılır):

[7] + [3] ⇒ [3, 7]

[3, 7] + [5] ⇒ [3, 5, 7]

[8] + [2] ⇒ [2, 8]

[3, 5, 7] + [2, 8] ⇒ [2, 3, 5, 7, 8]

[9] + [4] ⇒ [4, 9]

[15] + [6] ⇒ [6, 15]

[4, 9] + [6, 15] ⇒ [4, 6, 9, 15]

[2, 3, 5, 7, 8] + [4, 6, 9, 15] ⇒ [2, 3, 4, 5, 6, 7, 8, 9, 15]

Sonuç: 2, 3, 4, 5, 6, 7, 8, 9, 15

- Bölme $h = \log_2 n$ seviye; her seviyede n eleman üzerinden geçilir ⇒ $O(n \log n)$ (en iyi ve en kötü aynı: $\Theta(n \log n)$).
- **En büyük dezavantajı alan karmaşıklığı**: özyinelemeli çalıştığı için her bölmede fonksiyon kendi kopyasını oluşturur, bellek hızlı dolar [video 6 01:05:51].

```
Birlesmeli(D, ilk, son)
  if (ilk < son)
     p = (ilk + son) / 2          // asagi yuvarlanir
     Birlesmeli(D, ilk, p)
     Birlesmeli(D, p+1, son)
     Birlestir(D, ilk, p, son)

Birlestir(D, ilk, p, son)
  i = ilk;  j = p+1;  t = 0
  while (i <= p && j <= son)
     if (D[i] <= D[j]) G[t++] = D[i++]
     else              G[t++] = D[j++]
  while (i <= p)   G[t++] = D[i++]
  while (j <= son) G[t++] = D[j++]
  G'yi D[ilk..son]'a kopyala
```

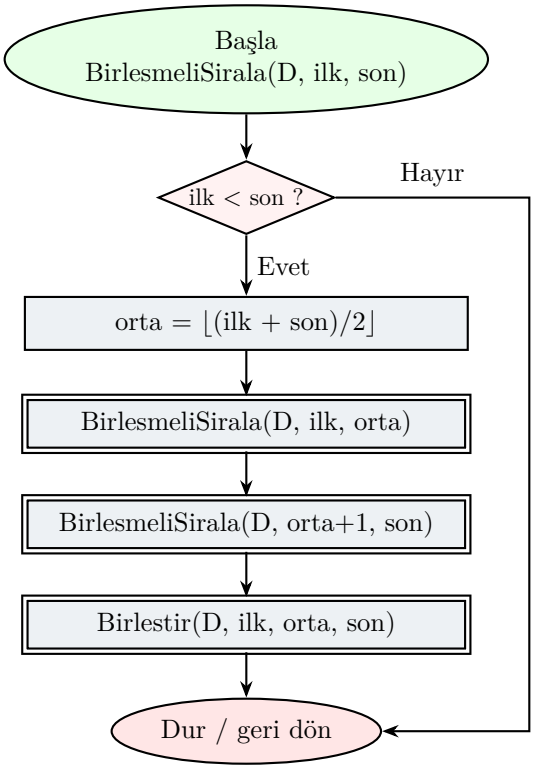
Zaman ve alan karmaşıklığı

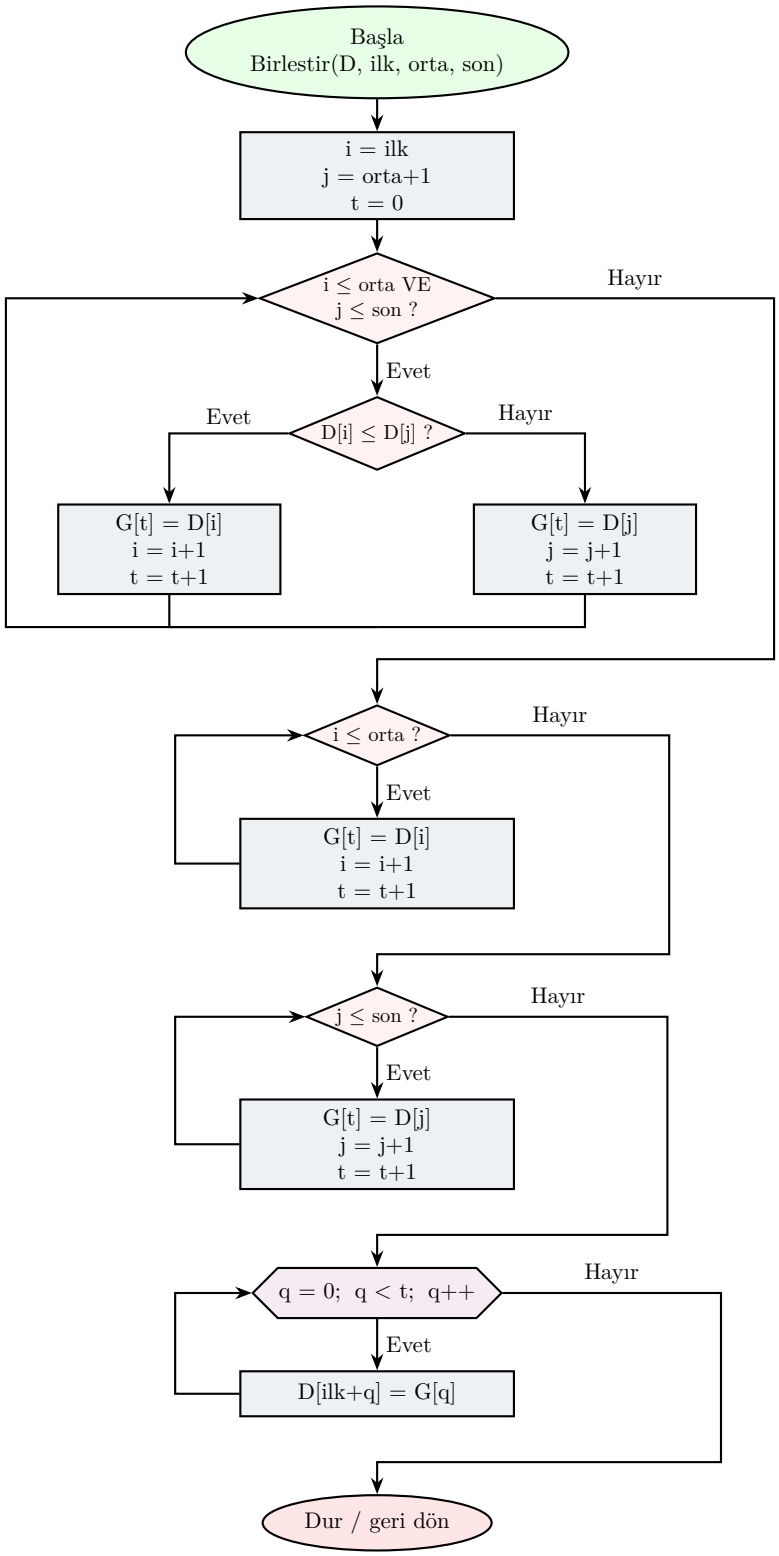
Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n \log n)$	bölme her zaman $\log_2 n$ seviye; her seviyede n eleman birleştirilir
En kötü	$O(n \log n)$	verinin geliş sırası bölmeyi değiştirmez
Ortalama	$\Theta(n \log n)$	en iyi ve en kötü aynı derecede
Ek alan	$O(n)$	geçici dizi `G` + özyinelemeli çağrıların kopyaları; hocanın vurguladığı dezavantaj

Birleşmeli – nihai özet

Özellik	En iyi durum	En kötü durum
Eleman karşılaştırması	$\approx n/2 \log_2 n$ (sıralı dizi)	$\approx n \log_2 n$
Kopyalama (G'ye ve geri)	$2n \log_2 n$	$2n \log_2 n$
$T(n)$	$T(n) = 2T(n/2) + cn \Rightarrow \approx cn \log_2 n$	
Zaman karmaşıklığı	$\Omega(n \log n)$	$O(n \log n) \Rightarrow \Theta(n \log n)$
Ek alan	$O(n)$	$O(n)$

Akış diyagramı: birleşmeli sıralama (ana fonksiyon ve birleştirme)





3.5 Kümeleme sıralama (Heap sort)

! Hoca derste vurguladı [video 6 Sıralama 3 44:25, 57:40]

İki aşaması vardır: A) hip ağacının oluşturulması, B) sıralama (koparma). *"Soru sorulduğu zaman bir kısmınız ağacı oluşturuyor ve bırakıyor. Sıralama işlemini gerçekleştirmiyor. O doğru cevap değil. Cevap eksik."* Ayrıca *"İkili arama ağacıyla karıştırmayın"* – hip ağacında küçükler sola, büyükler sağa kuralı yoktur.

Hip özelliği (ders varsayımı): her düğüm çocuklarına eşit veya onlardan *büyüktür* (max-heap); tersi de olabilir ama bir kural seçilip sonuna kadar uygulanır.

1. **Ekleme:** Zigzak mantığı – seviye seviye, soldan sağa (kök, sonra 2. kademe soldan sağa, sonra 3. kademe…). Her eklemeden sonra eklenen eleman ebeveyninden büyükse yer değiştirerek yukarı çıkar. *"Hip özelliği sağlanmıyorsa veri eklemeye devam edemeyiz... sonuç doğru çıksa da algoritma o şekilde çalışmaz."*
2. **Koparma:** Kök (en büyük) koparılır ve *kenara yazılır* (*"neyi nasıl sıraladığınızı göstermeniz gerekiyor"*). Zigzak sırasına göre en son eleman köke gelir ve *büyük olan çocukla* yer değiştirerek aşağı iner. Ağaç boşalana kadar devam edilir.

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

A) Hip (kümeleme) ağacının oluşturulması — zigzag (seviye seviye, soldan sağa) ekleme; her eklemede düğüm ≥ çocukları kontrolü:

7 eklendi; hip sağlanıyor



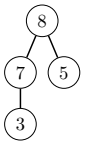
3 eklendi; hip sağlanıyor



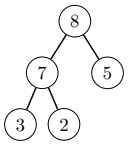
5 eklendi; hip sağlanıyor



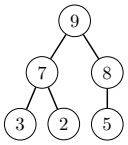
8 eklendi; 8 ↔ 3, 8 ↔ 7



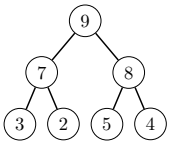
2 eklendi; hip sağlanıyor



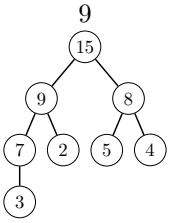
9 eklendi; 9 ↔ 5, 9 ↔ 8



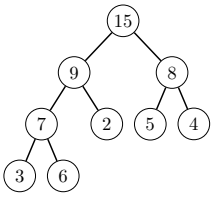
4 eklendi; hip sağlanıyor



15 eklendi; 15 ↔ 3, 15 ↔ 7, 15 ↔ 9

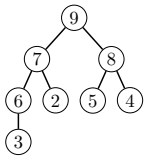


6 eklendi; hip sağlanıyor

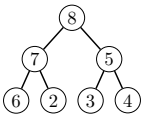


B) Sıralama (koparma) aşaması — kök koparılıp kenara yazılır, zigzag sırasına göre en son eleman köke gelir ve *büyük olan çocukla* yer değiştirerek aşağı iner:

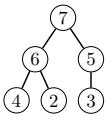
15 koparıldı, 6 köke geldi; 6 ↔ 9, 6 ↔ 7
Kopanlar: 15



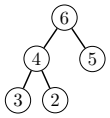
9 koparıldı, 3 köke geldi; 3 ↔ 8, 3 ↔ 5
Kopanlar: 15, 9



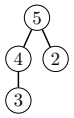
8 koparıldı, 4 köke geldi; 4 ↔ 7, 4 ↔ 6
Kopanlar: 15, 9, 8



7 koparıldı, 3 köke geldi; 3 ↔ 6, 3 ↔ 4
Kopanlar: 15, 9, 8, 7



6 koparıldı, 2 köke geldi; 2 ↔ 5
Kopanlar: 15, 9, 8, 7, 6



5 koparıldı, 3 köke geldi; 3 ↔ 4
Kopanlar: 15, 9, 8, 7, 6, 5



4 koparıldı, 2 köke geldi; 2 ↔ 3
Kopanlar: 15, 9, 8, 7, 6, 5, 4



3 koparıldı, 2 köke geldi
Kopanlar: 15, 9, 8, 7, 6, 5, 4, 3

2 koparıldı
Kopanlar: 15, 9, 8, 7, 6, 5, 4, 3, 2



Koparılanlar büyükten küçüğe çıktığı için sondan başa yazılır. **Sıralı dizi:** 2, 3, 4, 5, 6, 7, 8, 9, 15

Çıkmış sorular: 2020 Güz Vize 1a (sıralama), **2020 Güz Bütünleme 1a–c** (sıralama + akış diyagramı + yürütme zamanı) ve 2021 Yaz Final 1a–b (sıralama + akış diyagramı). Vize ve bütünleme çözümleri: `02_Vize_Cozumleri.pdf`. **Karmaşıklık:** ağaç yüksekliği $\log n$; her eleman için ekleme/koparma $O(\log n) \Rightarrow O(n \log n)$.

Kod (hocanın slayt stilinde, iki yardımcı fonksiyonla): Hoca sınavda genelde *"ya kaba kodu ya da akış diyagramı"* istiyor; kod, bölüm sonundaki akış diyagramlarıyla (`Kumeleme`, `HipEkle`, `KokSil`) birebir aynıdır.

```
Kumeleme ( int D[], int N)
{
    int H[N];                    // hip agaci, H[0] = kok
    int i, k;

    for (i=0; i<N; i++)          // A) AGAC OLUSTURMA
        HipEkle(H, i, D[i]);     // teker teker ekle

    for (k=N-1; k>=0; k--) {     // B) SIRALAMA
        D[k] = H[0];             // koku kopar, kenara yaz
        KokSil(H, k);            // sondakini koke koy
    }
}

HipEkle ( int H[], int son, int x)
{                                // x'i zigzag'a gore ekle
    int c, ebeveyn, gecici;
    H[son] = x;                  // ilk bos yer
    c = son;
    ebeveyn = (c-1)/2;
    while (c>0 && H[c]>H[ebeveyn]) {
        gecici = H[c];           // yukari tasi
        H[c] = H[ebeveyn];
        H[ebeveyn] = gecici;
        c = ebeveyn;
        ebeveyn = (c-1)/2;
    }
}

KokSil ( int H[], int son)
{                                // agac: H[0..son]
    int c, buyuk, gecici;
    H[0] = H[son];               // en sondaki koke
    son = son-1;
    c = 0;
    while (2*c+1 <= son) {       // cocugu var
        buyuk = 2*c+1;           // sol cocuk
        if (2*c+2 <= son && H[2*c+2] > H[buyuk])
            buyuk = 2*c+2;       // sag daha buyuk
        if (H[c] >= H[buyuk])    // hip saglandi
            break;
        gecici = H[c];           // BUYUK cocukla
        H[c] = H[buyuk];         // yer degistir
        H[buyuk] = gecici;
        c = buyuk;
    }
}
```

! Hoca derste vurguladı [video 2 Akış Diyagramı 58:33]

Kendi yazdığın fonksiyonu çağırıyorsan *"o fonksiyonun detaylarını veya akış diyagramını vermemiz gerekecek."* Yani `HipEkle` ve `KokSil`'in içi de yazılmalı; yalnızca ana fonksiyonu yazmak eksik cevaptır. Kelime sorusunda `>` yerine "alfabede sonra gelir" karşılaştırması kullanılır (birleşik alfabe: … P Q R S Ş T U Ü V W X Y Z).

Sık hatalar: küçük çocukla yer değiştirmek; eklerken `>=` kullanmak (eşitler gereksiz yer değiştirir); 0'dan başlayan indislerde ebeveyni `c/2` almak (doğrusu `(c-1)/2`); ders kitabındaki `BuildMaxHeap` (`for i = n/2 downto 1`) yöntemini yazmak – o yöntem ağacı hocanınkinden farklı kurar. Hip ağacında indisler (zigzag sırası): ebeveyn $(c-1)/2$, çocuklar $2c+1$ ve $2c+2$.

Zaman ve alan karmaşıklığı

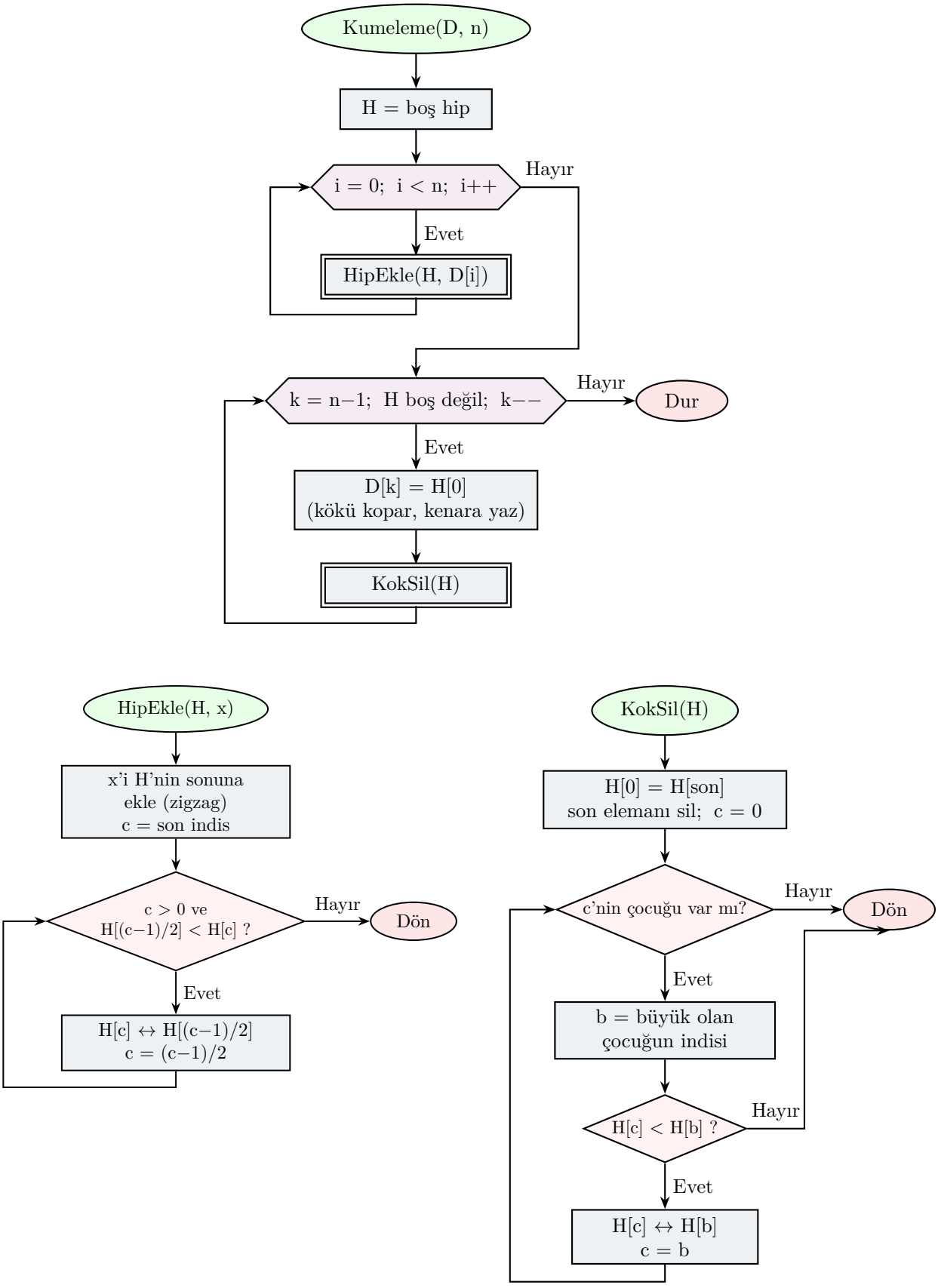
Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n \log n)$	her eleman için yukarı taşıma/aşağı indirme en fazla ağaç yüksekliği ($\log_2 n$) kadar
En kötü	$O(n \log n)$	n ekleme + n koparma, her biri $O(\log n)$
Ortalama	$\Theta(n \log n)$	en iyi ve en kötü aynı derecede
Ek alan	$O(n)$	hocanın yönteminde hip ayrı bir dizide (`H`) tutulur

Kümeleme – nihai özet

Özellik	En iyi durum	En kötü durum
A) Ağaç oluşturma (`HipEkle`)	n ekleme	n ekleme $\times \log_2 n$ yukarı taşıma
B) Koparma (`KokSil`)	n koparma	n koparma $\times \log_2 n$ aşağı indirme
$T(n)$		$\approx c_1 n \log_2 n + c_2 n \log_2 n$
Zaman karmaşıklığı	$\Omega(n \log n)$	$O(n \log n) \Rightarrow \Theta(n \log n)$
Ek alan	$O(n)$	$O(n)$

Akış diyagramı: kümeleme sıralama (iki aşama + iki yardımcı fonksiyon)

"Ağacı oluşturup bırakmak eksik cevaptır": akış diyagramında da *iki döngü* (oluşturma ve koparma) görünmelidir.



3.6 Hızlı sıralama (Quick sort)

Mantık: Bir *pivot* seçilir; pivottan küçükler sola, büyükler sağa geçer ve pivot *sıralamadaki yerini bulur*. Sonra sol ve sağ parçalara aynı işlem uygulanır. Birleşmeliden farkı: *"birleştirme adımı yok, bölerken sıralıyor"*.

! Hoca derste vurguladı [video 7 Sıralama 4 17:08, 35:41]

"Pivotu belirleme serbestliği hızlı sıralamada bize ait" (en sondaki, en baştaki, (ilk+son)/2, ortanca…). Ama *"bir sayıya odaklanmamak lazım"*; **genel geçer bir kural belirleyip bütün adımlarda uygulamamız lazım**. Sınav sorusunda kural verilir: 2020 "en sondaki kelimeyi pivot seçiniz", 2024 "en baştaki kelimeyi pivot olarak seçiniz". Eşit elemanlar pivotun yanına (genelde küçükler grubuna) alınır.

Tek dizi üzerinde yer değiştirme (hocanın yöntemi, pivot en sonda): soldan her eleman pivotla karşılaştırılır. Küçükse yerinde kalır. Büyükse 3 adım: (1) eleman pivotun yerine yazılır, (2) pivotun hemen önündeki eleman boşalan yere yazılır, (3) pivot bir sola kayar. Yeni gelen eleman tekrar kontrol edilir. Pivot ortadaysa sağındaki küçükler aynı mantıkla soluna alınır.

```
HizliSirala(D, ilk, son)
   if (ilk < son)
      p = Bolumle(D, ilk, son)
      HizliSirala(D, ilk, p-1)
      HizliSirala(D, p+1, son)

Bolumle(D, ilk, son)                  // pivot: en sondaki
   pivot = D[son];  p = son;  i = ilk
   while (i < p)
      if (D[i] > pivot)
         D[p] = D[i];  D[i] = D[p-1];  D[p-1] = pivot;  p = p-1
      else  i = i+1
   return p
```

▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

Pivot kuralı: dizinin (alt dizinin) EN SONUNDAKİ eleman pivot seçilir; bu kural *her adımda* aynen uygulanır. Pivottan büyükler sağa, küçükler sola geçer. Yer değiştirme hocanın tek dizi yöntemiyle yapılır: eleman pivotun yerine yazılır, pivotun yanındaki eleman boşalan yere yazılır, pivot bir birim kayar.

Başlangıç: 7, 3, 5, 8, 2, 9, 4, 15, 6

Alt dizi [0..8], pivot = **6**

7 > 6 olduğundan pivotun sağına: 15, 3, 5, 8, 2, 9, 4, **6**, 7

15 > 6 olduğundan pivotun sağına: 4, 3, 5, 8, 2, 9, **6**, 15, 7

8 > 6 olduğundan pivotun sağına: 4, 3, 5, 9, 2, **6**, 8, 15, 7

9 > 6 olduğundan pivotun sağına: 4, 3, 5, 2, **6**, 9, 8, 15, 7

Sonuç: [4, 3, 5, 2] **6** [9, 8, 15, 7] ⇒ 6 sıralamadaki yerini buldu (indis 4).

Alt dizi [0..3], pivot = **2**

4 > 2 olduğundan pivotun sağına: 5, 3, **2**, 4

5 > 2 olduğundan pivotun sağına: 3, **2**, 5, 4

3 > 2 olduğundan pivotun sağına: **2**, 3, 5, 4

Sonuç: [] **2** [3, 5, 4] ⇒ 2 sıralamadaki yerini buldu (indis 0).

Alt dizi [1..3], pivot = **4**

5 > 4 olduğundan pivotun sağına: 3, **4**, 5

Sonuç: [3] **4** [5] ⇒ 4 sıralamadaki yerini buldu (indis 2).

▷ [1..1] alt dizisinde tek eleman (3) kaldı ⇒ yerini buldu.

▷ [3..3] alt dizisinde tek eleman (5) kaldı ⇒ yerini buldu.

Alt dizi [5..8], pivot = **7**

9 > 7 olduğundan pivotun sağına: 15, 8, **7**, 9

15 > 7 olduğundan pivotun sağına: 8, **7**, 15, 9

8 > 7 olduğundan pivotun sağına: **7**, 8, 15, 9

Sonuç: [] **7** [8, 15, 9] ⇒ 7 sıralamadaki yerini buldu (indis 5).

Alt dizi [6..8], pivot = **9**

15 > 9 olduğundan pivotun sağına: 8, **9**, 15

Sonuç: [8] **9** [15] ⇒ 9 sıralamadaki yerini buldu (indis 7).

▷ [6..6] alt dizisinde tek eleman (8) kaldı ⇒ yerini buldu.

▷ [8..8] alt dizisinde tek eleman (15) kaldı ⇒ yerini buldu.

Sıralı dizi: 2, 3, 4, 5, 6, 7, 8, 9, 15

- **En kötü durum:** pivot hep en büyük ya da en küçük eleman (ör. sıralı dizi + son eleman pivot). Bölme olmaz, her seferinde $n-1$ eleman kalır ⇒ $T(n) = T(n-1) + cn$ ⇒ $O(n^2)$. *"Literatürdeki kaynaklara bakarsak da o şekildedir."*
- **En iyi durum:** pivot diziyi ortadan ikiye böler ⇒ $\log n$ seviye × n ⇒ $\Omega(n \log n)$.
- İyi pivot için öneriler: ortanca (ama bulmak için sıralama gerekir), ortalamaya en yakın değer, frekans/istatistik.
- Dezavantajlar: uygun pivot seçme zorluğu ve özyinelemeden dolayı yüksek alan maliyeti.
- Satır satır yürütme zamanının tam çözümü: `02_Vize_Cozumleri.pdf`, 2024–25 Vize Soru 2b.

Zaman ve alan karmaşıklığı

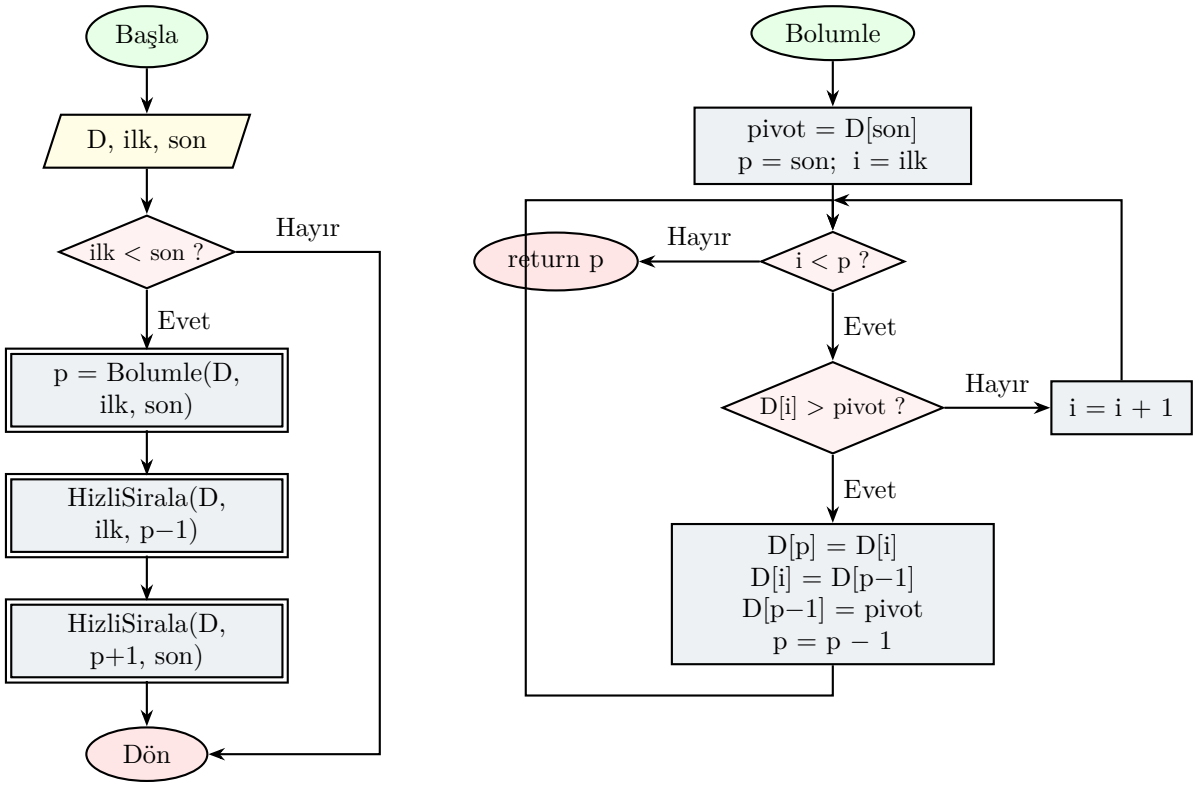
Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(n \log n)$	pivot her seferinde diziyi ortadan ikiye böler: $\log_2 n$ seviye × n
En kötü	$O(n^2)$	pivot hep en büyük/en küçük (ör. sıralı dizi + sondaki pivot): her seferinde $n-1$ eleman kalır
Ortalama	–	en iyi $n \log n$, en kötü n^2 dereceli ⇒ Θ verilmez
Ek alan	$O(\log n)$ – $O(n)$	dizi yerinde bölünür ama özyineleme derinliği kadar fonksiyon kopyası (en iyi $\log n$, en kötü n)

Hızlı – nihai özet

Özellik	En iyi durum	En kötü durum
Bölümleme $T_B(m)$ (m eleman)	$3m+2$	$6m-1$
Eleman karşılaştırması (`D[i] > pivot`)	$\approx n \log_2 n$	$n(n-1)/2$
$T(n)$	$\approx 6n \log_2 n$	$\approx 3n^2 + 4n$
Zaman karmaşıklığı	$\Omega(n \log n)$	$O(n^2)$
Ek alan	$O(\log n)$	$O(n)$

Akış diyagramı: hızlı sıralama (ana fonksiyon ve bölümleme)

Son iki vizede üst üste soruldu. “Pivot en baştaki” denirse pivotun sağındakiler soldan sağa taranır, küçükler sola alınır.



3.7 Radiks sıralama (Radix sort)

Mantık: En önemsiz haneden (sayıda birler basamağı, kelimede *son harf*) başlanır; elemanlar o hanedeki değere göre *kovalara* (sayılarda 0–9 = 10 kova; Türkçe kelimelerde 29+ harf kadar kova) *geliş sırasıyla* dağıtılır, kovalar sırayla toplanır. Geçiş sayısı = en uzun elemanın hane sayısı.

- **Hane eşitleme:** Sayılarda değeri değiştirmemek için *başa 0* eklenir (87 → 087). Kelimelerde sözlük sırasını bozmamak için *sona* A eklenir (öğrenci aktarımı: *“radix için hoca kelimenin en sağına A ekleyip…”* – *“Sona”*; 2021 çözümü: QOAW, HTRA, LMİA, UAAA…). Kısa kelimeler aynı başlayan uzunlardan önce gelir.
- **Sayı örneği (ders):** 275, 087, 426, 061, 509, 170, 677, 503
  1. geçiş (birler): 170, 061, 503, 275, 426, 087, 677, 509
  2. geçiş (onlar): 503, 509, 426, 061, 170, 275, 677, 087
  3. geçiş (yüzler): **061, 087, 170, 275, 426, 503, 509, 677**

Kelime örneği (ders: Ahmet, Ali, Ayşe, Veli, Mehmet, Serkan):

Hane eşitleme (en uzun kelime 6 harf; sözlük sırası bozulmasın diye kelimelerin **sonuna** ‘A’ eklenir): AHMETA, ALİAAA, AYŞEAA, VELİAA, MEHMET, SERKAN

1. geçiş (sağdan 1. harfe göre kovalar): **A**: AHMETA, ALİAAA, AYŞEAA, VELİAA; **N**: SERKAN; **T**: MEHMET

⇒ AHMETA, ALİAAA, AYŞEAA, VELİAA, SERKAN, MEHMET

2. geçiş (sağdan 2. harfe göre kovalar): **A**: ALİAAA, AYŞEAA, VELİAA, SERKAN; **E**: MEHMET; **T**: AHMETA

⇒ ALİAAA, AYŞEAA, VELİAA, SERKAN, MEHMET, AHMETA

3. geçiş (sağdan 3. harfe göre kovalar): **A**: ALİAAA; **E**: AYŞEAA, AHMETA; **İ**: VELİAA; **K**: SERKAN; **M**: MEHMET

⇒ ALİAAA, AYŞEAA, AHMETA, VELİAA, SERKAN, MEHMET

4. geçiş (sağdan 4. harfe göre kovalar): **H**: MEHMET; **İ**: ALİAAA; **L**: VELİAA; **M**: AHMETA; **R**: SERKAN; **Ş**: AYŞEAA

⇒ MEHMET, ALİAAA, VELİAA, AHMETA, SERKAN, AYŞEAA

5. geçiş (sağdan 5. harfe göre kovalar): **E**: MEHMET, VELİAA, SERKAN; **H**: AHMETA; **L**: ALİAAA; **Y**: AYŞEAA

⇒ MEHMET, VELİAA, SERKAN, AHMETA, ALİAAA, AYŞEAA

6. geçiş (sağdan 6. harfe göre kovalar): **A**: AHMETA, ALİAAA, AYŞEAA; **M**: MEHMET; **S**: SERKAN; **V**: VELİAA

⇒ AHMETA, ALİAAA, AYŞEAA, MEHMET, SERKAN, VELİAA

Eklenen A'lar atılır. **Sonuç:** AHMET, ALİ, AYŞE, MEHMET, SERKAN, VELİ

Karmaşıklık: d hane, n eleman, k kova: $O(d \cdot (n+k))$; d sabit kabul edilirse doğrusal.

```
RadiksSirala(D, n)
   d = en uzun elemanin hane sayisi
   tum elemanlari d haneye esitle        // sayida basa 0, kelimede sona A
   for (h = d; h >= 1; h--)              // sagdan sola her hane
      kovalari bosalt
      for (i = 0; i < n; i++)
         kova[ D[i]'nin h. hanesi ].enqueue(D[i])
      j = 0
      for (her kova c, alfabe/rakam sirasiyla)
         while (!kova[c].isEmpty())  D[j++] = kova[c].dequeue()
   eklenen 0/A karakterlerini at
```

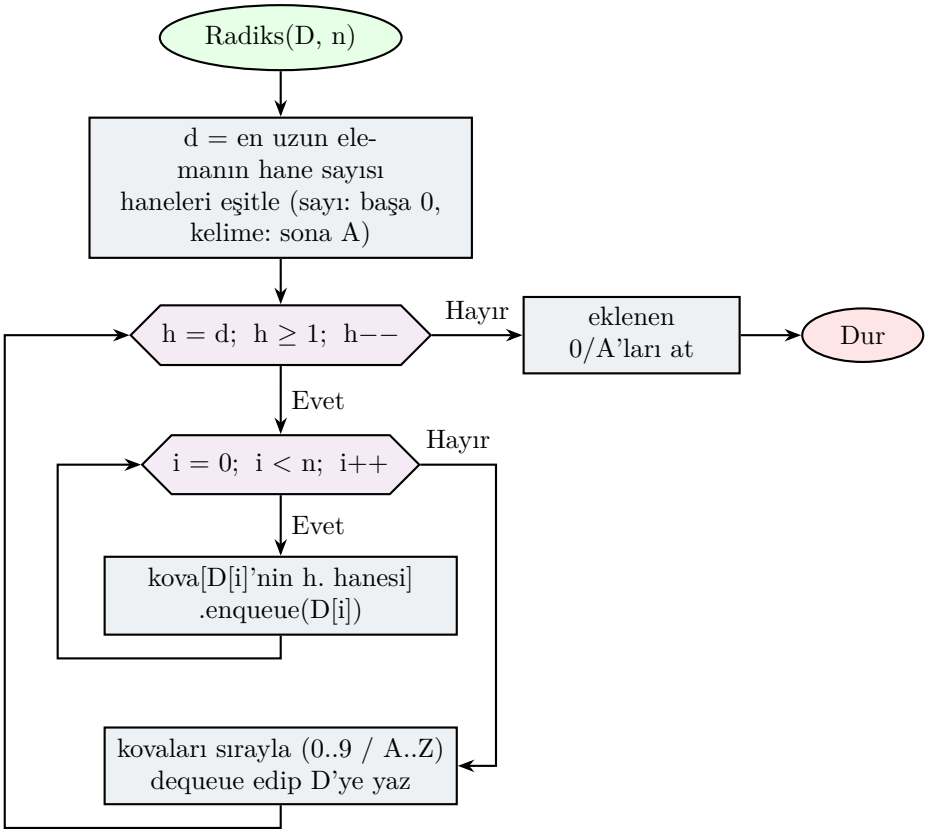
Zaman ve alan karmaşıklığı

Durum	Karmaşıklık	Ne zaman / not
En iyi	$\Omega(d(n+k))$	karşılaştırma yapılmaz; her geçişte n eleman dağıtılır, k kova toplanır
En kötü	$O(d(n+k))$	verinin sırası iş miktarını değiştirmez; d = en uzun elemanın hane sayısı
Ortalama	$\Theta(d(n+k))$	en iyi ve en kötü aynı; d sabitse doğrusal
Ek alan	$O(n+k)$	kovalar (kuyruklar) bütün elemanları geçici olarak tutar

Radiks – nihai özet

Özellik	En iyi durum	En kötü durum
Eleman karşılaştırması	0	0
Kovaya dağıtma (`enqueue`)	$d \cdot n$	$d \cdot n$
Kovadan toplama (`dequeue`)	$d \cdot n$	$d \cdot n$
$T(n)$	$\approx c\,d\,(n+k)$	
Zaman karmaşıklığı	$\Theta(d(n+k))$	
Ek alan	$O(n+k)$	$O(n+k)$

Akış diyagramı: radiks sıralama



▷ Bu bölümün adım adım oynatılabilen etkileşimli animasyonu HTML sürümündedir.

3.8 Özet tablo

Algoritma	En iyi	En kötü	Θ	Ek alan	Not
Araya sokma	$\Omega(n)$	$O(n^2)$	–	$O(1)$	en iyi: sıralı; en kötü: ters sıralı veya hepsi eşit
Seçmeli	$\Omega(n^2)$	$O(n^2)$	$\Theta(n^2)$	$O(1)$	hoca kodu: en iyi ters sıralı/hepsi eşit
Kabarcık	$\Omega(n^2)$	$O(n^2)$	$\Theta(n^2)$	$O(1)$	en iyi: sıralı/hepsi eşit
Birleşmeli	$\Omega(n \log n)$	$O(n \log n)$	$\Theta(n \log n)$	$O(n)$	alan maliyeti yüksek
Kümeleme	$\Omega(n \log n)$	$O(n \log n)$	$\Theta(n \log n)$	$O(n)$	iki aşama!
Hızlı	$\Omega(n \log n)$	$O(n^2)$	–	$O(\log n)$–$O(n)$	pivot seçimi belirleyici
Radiks	$O(d(n+k))$		$\Theta(d(n+k))$	$O(n+k)$	karşılaştırmasız, kovalar